



Intro

We're changing skins for objects in game!

All the objects in game are divided into different types of files: Meshes, Textures and Materials (the collective textures into one solid file). When we want to look for what textures belong to each mesh, we must inspect the mesh or BP, which will then tell us where the skin is coming from. It will most likely link to a **Material Instance (M)**, where the **Textures (T)** are listed, and this is where we'll know which files we want to grab.

So, the goal is to take an object in game and find its textures and make the changes we want. We can do this by either giving our new texture the same name as the old one, meaning our custom file will override the vanilla file, or we can give it a brand-new name, and then we have to make sure that any meshes or materials that call for the old name get switched to the new name.



UV-mapping

Meshes are “UV-mapped” onto textures. You “unwrap” your mesh into smaller parts and then map them onto your texture image so the “islands” (surfaces) line up with the colors you want. Icarus meshes are already UV-mapped, so we just need to find the right texture and area that we want to change.

Materials are divided into different layers which the connected program adds together to form a finished image.

Albedo (A) is the color layer. This is where you put in the colors of the object and then map the unwrapped mesh onto the image, so it fits with shapes and colors. It’s about visualizing a flat image wrapping around the mesh, so the placement of the UV shapes on the image is important. If you place the mesh for a face onto an area meant for pants, your character will end up with a denim-colored face.



Normal maps (N) are artificial textures and depth on an otherwise flat object. It can make the object look patterned without making any changes to the mesh itself, saving a lot of time and processing power on delicate modeling. You don’t need to sculpt a complex brick wall with individual bricks and uneven edges – you can simply “paint” on the details.

Normal textures are typically a light purple-ish color with iridescent markings that determine the pattern and shading direction.





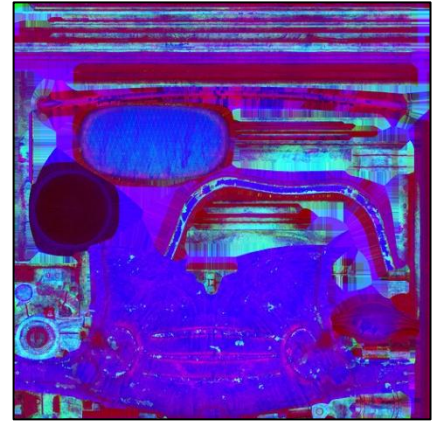
Roughness, Metallic, Ambient Occlusion (RMA) is a combined texture map describing the levels of three different properties:

✦ **Roughness** means how shiny or matte an object is

✦ **Metallic** is the metallic sheen

✦ **Ambient Occlusion** means the light/shadow depth

RMAs may look like different combinations of reds, purples, blues and greens.



Some texture files may have other letters or colors on them; these are different types of consolidated textures but will still be important. We won't need to understand them or sort them out, we just want to make sure to include them and keep the changes uniform.

For things like posters, they're meant to be flat images, so that's an example of something that likely will only need to be edited in the color layer.

When editing a skin, we want to edit all the texture images that belong to that object. For example, if we take an envirosuit and put a custom patch on the color layer, it will look flat on the actual character. That is why we want to also edit the normal map and possibly the other layers to give it some texture and shading, so the new addition will look natural and part of the object.

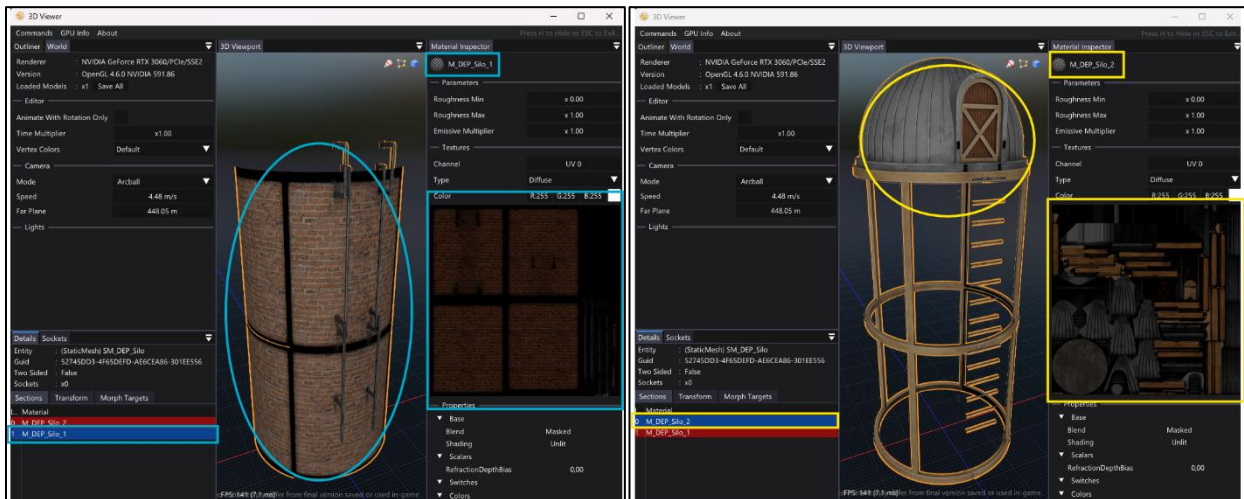


Programs

FModel + UAssetGUI

✎ This is where the extraction happens and where we need to pay attention. First, we need to index our object's connected files, so we know what we're dealing with.

- ✎ For example, if we want to edit the Homestead Silo, we can find it under Icarus/Content/ASS/DEP/DEP_Silo where we find: StaticMesh, DestructibleMesh (the mesh of the object when it breaks), six Textures and two Material Instances (a "shader" setup telling Unreal how to use and combine textures together).
- ✎ The reason silo has two materials instead of one can be found if we open the SM inside FModel and inspect it in the 3D viewer.



✎ Here we can see that the silo's skin is divided into two parts; one for bricks, and one for wood and roof. The UVs are already mapped on the object, so for the silo it means that the materials for the roof and the bricks are already separated and lined up with the textures. We just want to make sure we have the right one when we're editing.

Icarus Modding – Textures

Documentation by Ferani



✦ If we want, we can inspect the SMs `.uasset` file in UAssetGUI and see where it draws its skin from.

Objects tend to be grouped with their relevant files so they're easy to keep track of, but not always, so you want to make sure you know where to find them.

The Import Data shows us that the Silo SM uses M_DEP_Silo_1+2, which are the ones included in the DEP_Silo folder, so that makes sense.

	OuterIndex	ObjectName
	-10	Default__BodySetup
	-10	BodySetup
	-10	StaticMesh
	-11	NavCollision
ceConstant	-8	M_DEP_Silo_1
ceConstant	-9	M_DEP_Silo_2
	-11	Default__NavCollision
	0	/Game/ASS/DEP/DEP_Silo/M_DEP_Silo_1
	0	/Game/ASS/DEP/DEP_Silo/M_DEP_Silo_2
	0	/Script/Engine
	0	/Script/NavigationSystem
	-10	Default__StaticMesh

✦ If we then inspect one of the Material Instances, we should see the three texture files (one for each type of texture) that match the material number, and for M_DEP_Silo_1 we do indeed see the three textures that belong.

File	Edit	View	Utils	Help		ClassPackage	ClassName	OuterIndex	ObjectName
General Information						/Script/CoreUObject	Package	0	/Game/ASS/CHA/Mat/Default/T_Master_Default_RGBMask
Name Map						/Script/CoreUObject	Package	0	/Game/ASS/CHA/Mat/LIB/TX/skin_micro_n
Import Data						/Script/CoreUObject	Package	0	/Game/ASS/DEP/DEP_Silo/T_DEP_Silo_1_A
Export Information						/Script/CoreUObject	Package	0	/Game/ASS/DEP/DEP_Silo/T_DEP_Silo_1_N
Custom Version Container						/Script/CoreUObject	Package	0	/Game/ASS/DEP/DEP_Silo/T_DEP_Silo_1_RMA
Export Data									

✦ Finally, we'll inspect one of the texture `.uassets` and see what it looks like inside.

File	Edit	View	Utils	Help		Name	Type	Variant	Value	Value 2	Value 3
General Information					0	ImportedSize	IntPoint		4096	4096	
Name Map											
Import Data											
Export Information											
Custom Version Container											
Export Data											
Export 1 (T_DEP_Silo_1_A)											
Texture2D (3)											
ImportedSize (1)											
LightingGuid (1)											
Extra Data (3244 B)											

We cannot see any visual representation of the texture in UAssetGUI; only some metadata, and even that is limited. But what's important is looking at the **"ImportedSize"** and remembering the dimensions (measured in pixels), because we want to make sure that our new texture has the same dimensions.



The size may vary between the **Albedo (A)**, **Normal (N)** and **RMA** files, even on the same object, so it's a good idea to check each of them.

- ✦ If you want a visual of the UV-map itself, you can export the SM to Blender and open it. Here you can see how the mesh has been unwrapped onto a flat plane, and if you import the texture image, you can view the “islands” on the UV map/atlas.



Unreal Editor 4.27.2

This is a vital part of this process, because the game doesn't care about PNGs on their own, and they will not be treated as valid image files when importing. Instead we need UE to "cook" the PNG for us, meaning it will solidify the data into an acceptable format for the game.

- ✎ Install Unreal Editor and make sure that the version is set to 4.27.2, or else the final assets will be incompatible.
- ✎ Start a new empty project and name it whatever you want, that is not important, as long as you remember *where* you saved it.

Inside of the project folder is a "Content"-folder, where we deposit all our files during the process, and the cooked files will be somewhere like:

‣ Project_Name\Saved\Cooked\WindowsNoEditor\Project_Name\Content

UE will import PNG images as textures, and it may recognize **N** as the normal map right away.



DataTable

When changing textures, we may not even need to write anything in the data-section of our mod. We can just fill in the meta-data of our mod, recreate the folder structure from the source textures, and then move along to the next step.

```
Current Mod Name: My Silo

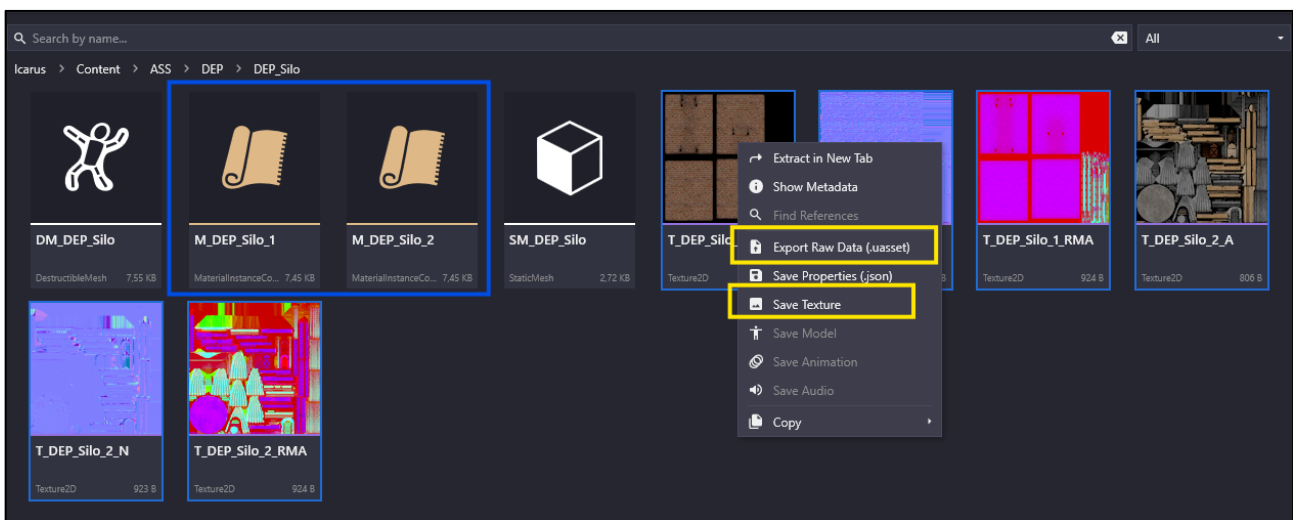
Top Search Find/Replace Clean up JSON

1 {
2   "name": "My Silo",
3   "author": "Ferani",
4   "version": "1.0",
5   "week": "",
6   "description": "Recolor of the Homestead Silo.",
7   "fileName": "My_Silo",
8   "readmeURL": "",
9   "imageUrl": "",
10  "Level2": "True",
11  "Rows": [
12    {
13      "CurrentFile": "EndOfMod"
14    }
15  ]
16 }
17
```



FModel

- ✎ Take the six Textures and right-click them
 - ✎ We want to save them as both `.uassets` and then also as Textures (PNG)
 - ✎ Keep an eye on the resolution/size of the image to keep it the same as the original
- ✎ Take the Material Instances extract them as regular `.uassets`. They are not image files themselves, but that's okay since we're only interested in the data on how those images interact



- ✎ These files will land in an FModel export-folder, make sure you know where it is

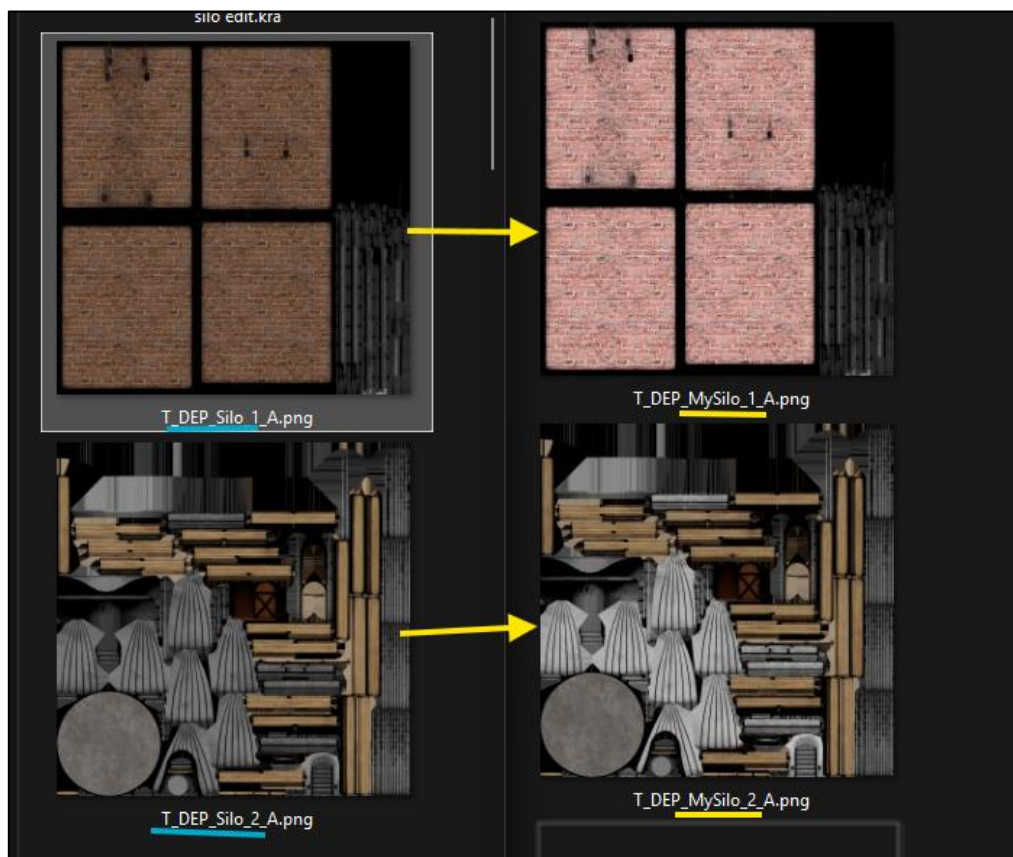


Texture editing

Now we'll edit the images as we like!

- ✎ Open an image editing program of your choice, import the PNGs, and make changes to the texture as you like.
 - ✎ If it's a simple color swap on something, you likely won't need to change the normal or RMA maps, so you can leave those be
 - ✎ Keep in mind that the image is a flat representation of the mesh, so knowing where the seams and edges are mapped is a good idea for more complex edits
 - ✎ If you're adding something that affects the texture of the image, it's very helpful to have an AI assistant when making the changes to all three files at once, as it will help make sure the changes are uniform, so you don't have to do guesswork on where the layers will join together again

We need to make sure that the finished images are the same resolutions as their original .uasset textures **before** they're imported into Unreal Editor. The images may still be properly made into .uassets, but the game won't recognize it if the dimensions aren't the same.





Unreal Editor 4.27.2

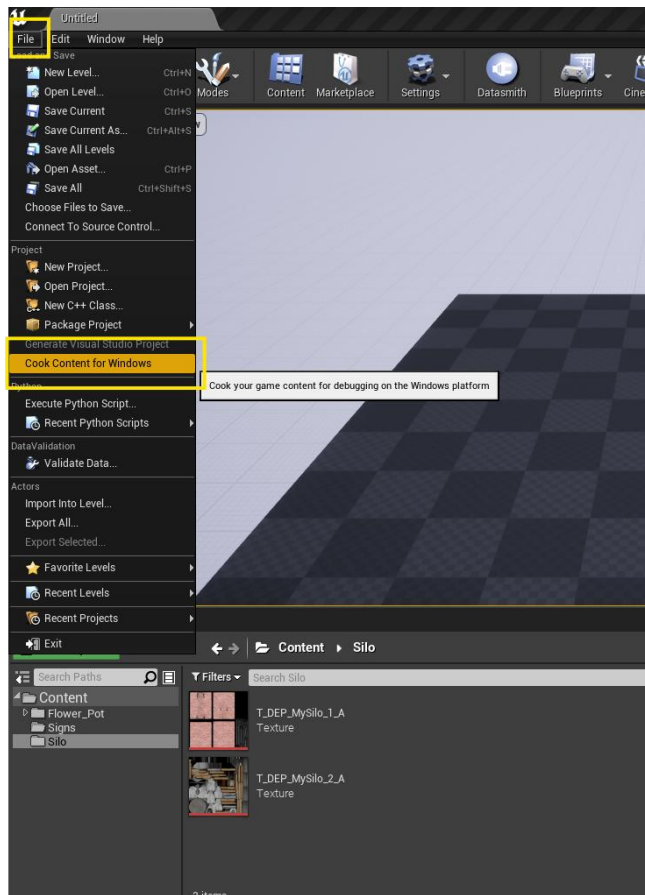
✎ Open your UE project and drag your texture PNGs into the “Content” area in the program (or place them in the project folder under “Content”)

✎ Give them new names, such as T_DEP_MySilo_1+2 but keep the **A/N/RMA** names at the end so we can remember which is which

✎ OR you can keep the original names – this will force the game to override the original files with our own, and we won’t have to change the names in the material files

✎ When you have your new texture files imported into the UE content-folder, go to the top left of the window, press “File” and then “Cook content”

✎ It means that every file you have inside of UE at that time will be processed into finished **.uasset** files that Icarus understands



✎ Now go to the cooked files folder and grab your textures, including any **.uexp** and **.ubulk** it may have generated as well

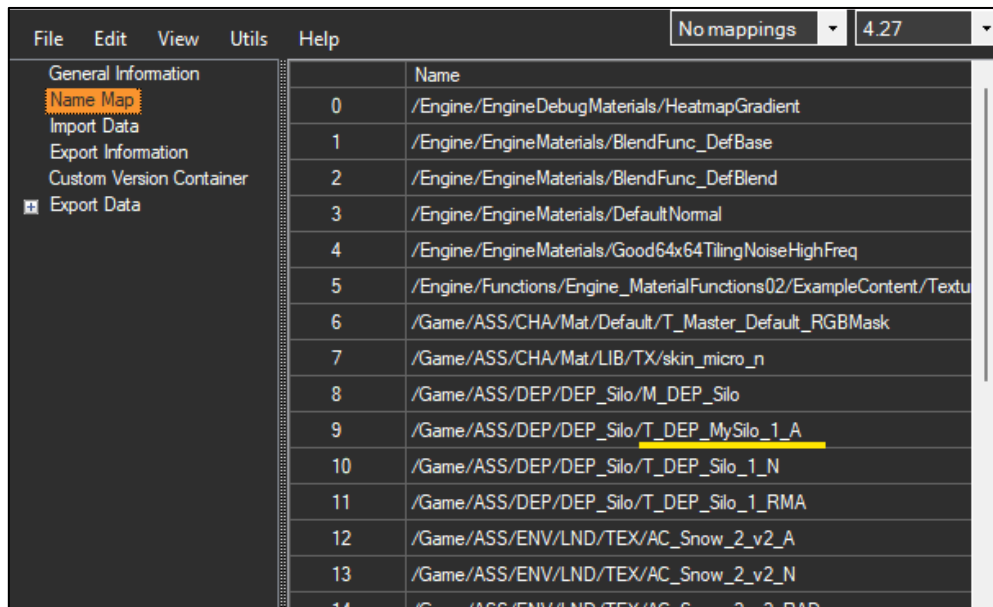
✎ Open your mod folder and recreate the directory for the original texture files. In our case:

✎ Extracted_Mods/My_Silo/ASS/DEP/DEP_Silo

✎ Place the newly cooked files in here



- ✎ Open the material files and look through the **Name Map** for instance of the textures you need to replace
- ✎ Replace any instance of “T_DEP_Silo” with the corresponding “T_DEP_**MySilo**” so all of them are now pointing to our new textures



Now we should be able to open the game and see the changed texture!

